Solution: Reverse a String

Problem Statement

Given a string, write a function that uses a stack to reverse the string. The function should return the reversed string.

Examples

Example 1:

```
Input: "Hello, World!"
Output: "!dlroW ,olleH"
```

Example 2:

```
Input: "OpenAI"
Output: "IAnepO"
```

Example 3:

```
Input: "Stacks are fun!"
Output: "!nuf era skcatS"
```

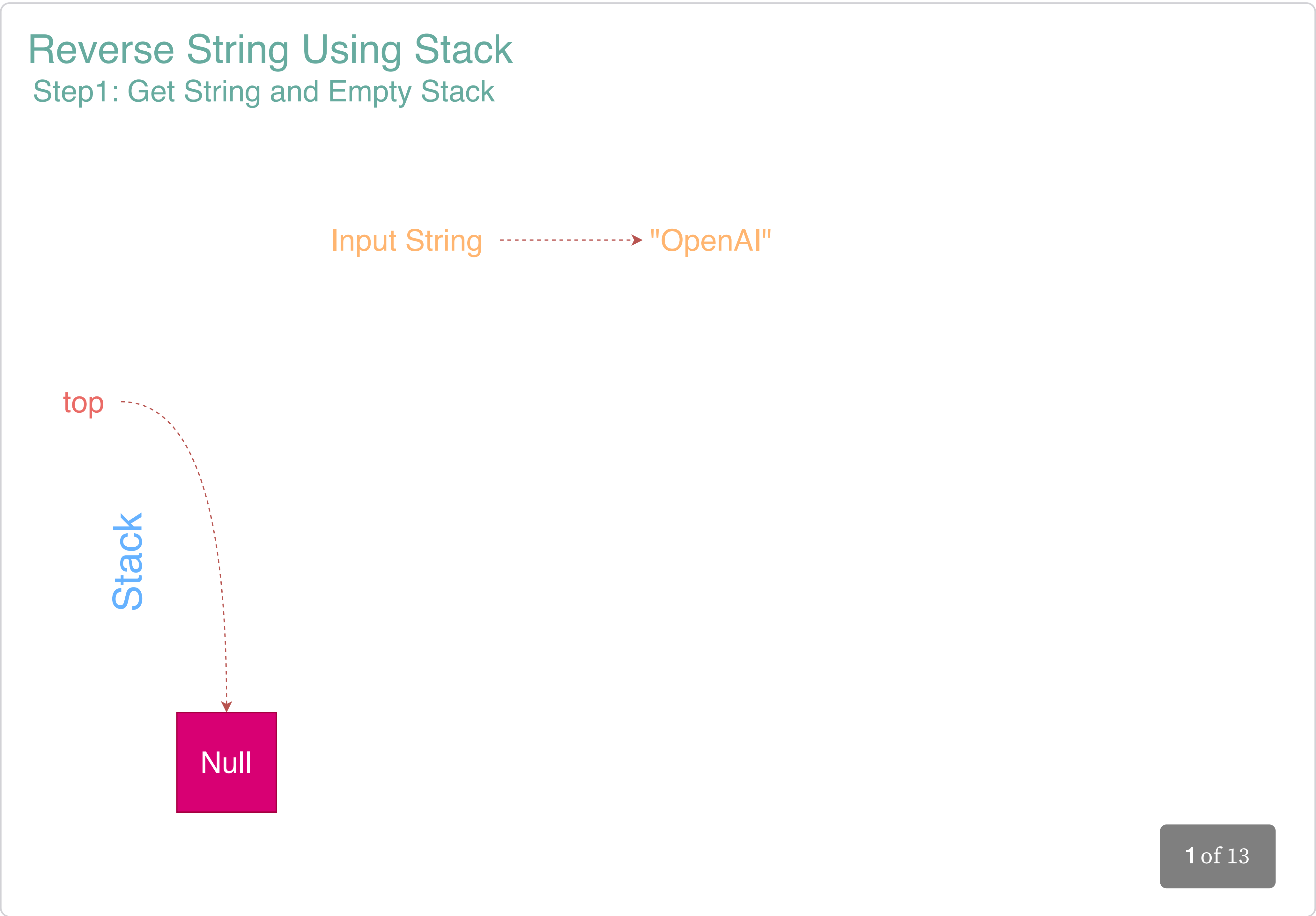
Solution

The solution to reverse a string can be elegantly achieved using a stack. The algorithm involves pushing each character of the string onto a stack and then popping them off, which naturally reverses their order. As we iterate through the string, each character is added to the top of the stack.

Once the entire string has been processed, we remove the characters from the stack one by one and append them to a new string. This process ensures that the characters are appended in reverse order, as the last character pushed onto the stack will be the first to be popped off. This method efficiently reverses the string while maintaining the integrity of the original data.

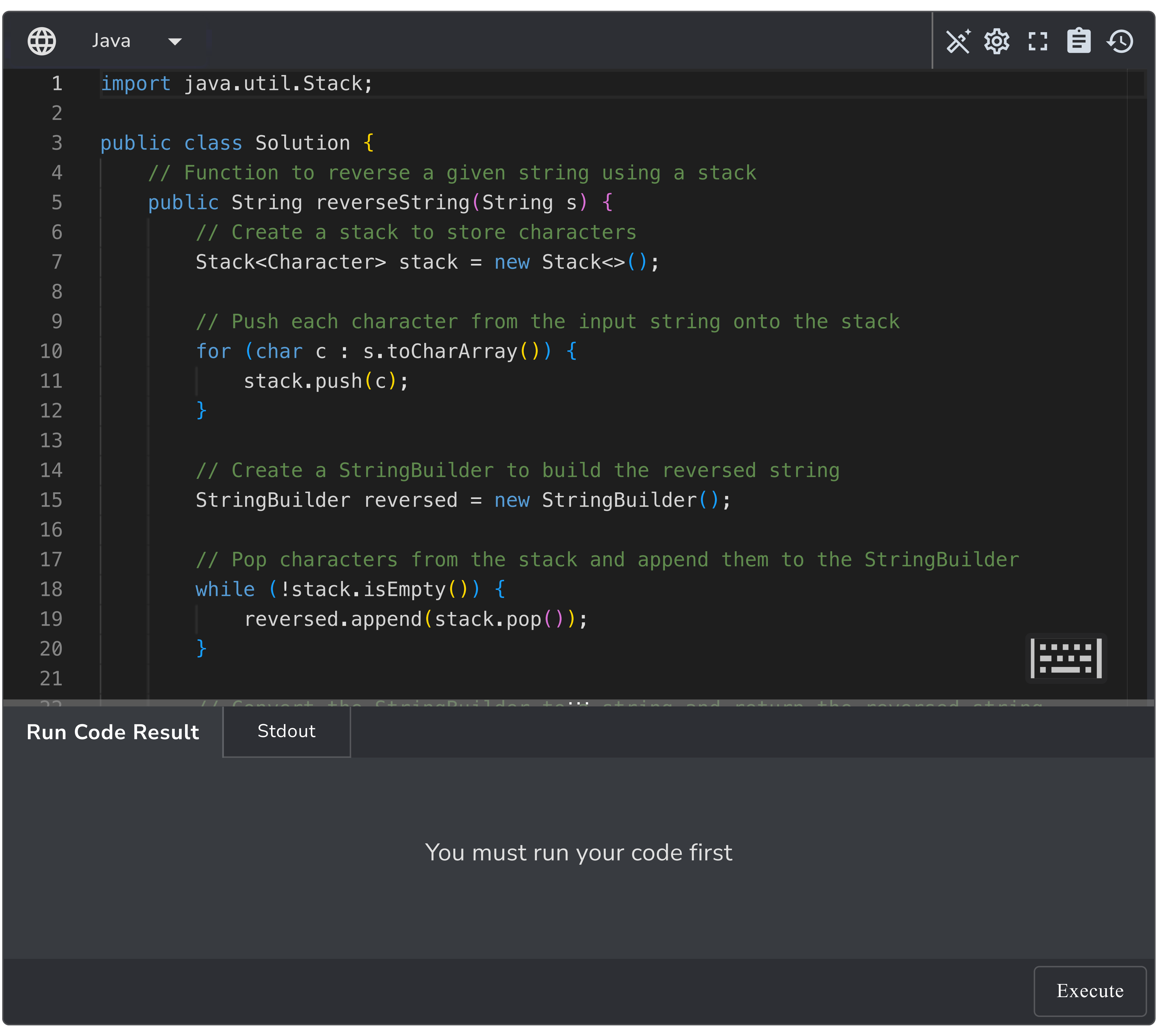
Here is the step-by-step algorithm.

1. Initialize an empty stack.
2. For each character in the input string, push the character into the stack.
3. Initialize an empty string to hold the reversed string.
4. While the stack is not empty, pop out the top character from the stack and append it to the reversed string.
5. Finally, return the reversed string.



Code

Here is how we can implement this algorithm:



```java
import java.util.Stack;

public class Solution {
    // Function to reverse a given string using a stack
    public String reverseString(String s) {
        // Create a stack to store characters
        Stack<Character> stack = new Stack<>();

        // Push each character from the input string onto the stack
        for (char c : s.toCharArray()) {
            stack.push(c);
        }

        // Create a StringBuilder to build the reversed string
        StringBuilder reversed = new StringBuilder();

        // Pop characters from the stack and append them to the StringBuilder
        while (!stack.isEmpty()) {
            reversed.append(stack.pop());
        }

```

You must run your code first

Time and Space Complexity:

Time Complexity: $O(n)$, where n is the length of the input string. This is because we iterate through the string once to push all characters into the stack and then another time to pop all characters out of the stack.

Space Complexity: $O(n)$, where n is the length of the input string. This is because we use a stack to hold all characters of the string.

← Previous
Problem 2: Reverse A String

Next →
Problem 3: Decimal To Binary Conversion

☐ Mark as Completed